

AVL AĞAÇLARI

ESMA BALIKÇI

24080410019

```
namespace veriyapilari6
{
    internal class Program
    {
        //AVL Ağacında ekleme ve silme (denge koruma) arama ve listeleme
        class Node
        {
            public int Value;
            public Node Left;
            public Node Right;
            public int Height;

            public Node(int value)
            {
                Value = value;
                Left = Right = null;
                Height = 1;
            }
        }

        class AVLTree
        {
            private Node root;
            public AVLTree()
            {
                root = null;
            }

            // Ağaçta ekleme işlemi
            public void Insert(int value)
            {
                root = InsertRec(root, value);
            }

            private Node InsertRec(Node node, int value)
            {
                if (node == null)
                    return new Node(value);

                if (value < node.Value)
                    node.Left = InsertRec(node.Left, value);
                else if (value > node.Value)
                    node.Right = InsertRec(node.Right, value);
                else
                    return node; // Aynı değerler eklenmesin
                                // Yükseklik güncellemesi

                node.Height = 1 + Math.Max(GetHeight(node.Left), GetHeight(node.Right));

                // Dengeyi kontrol et
                int balance = GetBalance(node);

                // Denge bozulmuşsa, uygun rotasyonu yap
                if (balance > 1 && value < node.Left.Value)
                    return RightRotate(node);
            }
        }
    }
}
```

```

        if (balance < -1 && value > node.Right.Value)
            return LeftRotate(node);

        if (balance > 1 && value > node.Left.Value)
        {
            node.Left = LeftRotate(node.Left);
            return RightRotate(node);
        }

        if (balance < -1 && value < node.Right.Value)
        {
            node.Right = RightRotate(node.Right);
            return LeftRotate(node);
        }

        return node;
    }

    private Node RightRotate(Node y)
    {
        Node x = y.Left;
        Node T2 = x.Right;

        // Rotasyonu yap
        x.Right = y;
        y.Left = T2;

        // Yükseklikleri güncelle
        y.Height = Math.Max(GetHeight(y.Left), GetHeight(y.Right)) + 1;
        x.Height = Math.Max(GetHeight(x.Left), GetHeight(x.Right)) + 1;

        return x;
    }

    // Sol rotasyon
    private Node LeftRotate(Node x)
    {
        Node y = x.Right;
        Node T2 = y.Left;

        // Rotasyonu yap
        y.Left = x;
        x.Right = T2;

        // Yükseklikleri güncelle
        x.Height = Math.Max(GetHeight(x.Left), GetHeight(x.Right)) + 1;
        y.Height = Math.Max(GetHeight(y.Left), GetHeight(y.Right)) + 1;

        return y;
    }

    // Bir düğümün yüksekliğini al
    private int GetHeight(Node node)
    {
        return node == null ? 0 : node.Height;
    }

    // Düğümün dengesini al
    private int GetBalance(Node node)
    {
        return node == null ? 0 : GetHeight(node.Left) - GetHeight(node.Right);
    }

```

```

// Ağaçta arama işlemi
public bool Search(int value)
{
    return SearchRec(root, value);
}

private bool SearchRec(Node node, int value)
{
    if (node == null)
        return false;

    if (value < node.Value)
        return SearchRec(node.Left, value);
    else if (value > node.Value)
        return SearchRec(node.Right, value);
    else
        return true; // Bulundu
}

// Ağaçta silme işlemi
public void Delete(int value)
{
    root = DeleteRec(root, value);
}

private Node DeleteRec(Node root, int value)
{
    if (root == null)
        return root;

    if (value < root.Value)
        root.Left = DeleteRec(root.Left, value);
    else if (value > root.Value)
        root.Right = DeleteRec(root.Right, value);
    else
    {
        if (root.Left == null || root.Right == null)
        {
            Node temp = root.Left != null ? root.Left : root.Right;

            if (temp == null)
            {
                temp = root;
                root = null;
            }
            else
                root = temp;
        }
        else
        {
            Node temp = GetMinValueNode(root.Right);
            root.Value = temp.Value;
            root.Right = DeleteRec(root.Right, temp.Value);
        }
    }
    if (root == null)
        return root;

    root.Height = Math.Max(GetHeight(root.Left), GetHeight(root.Right)) + 1;

    int balance = GetBalance(root);
}

```

```

        if (balance > 1 && GetBalance(root.Left) >= 0)
            return RightRotate(root);

        if (balance > 1 && GetBalance(root.Left) < 0)
        {
            root.Left = LeftRotate(root.Left);
            return RightRotate(root);
        }

        if (balance < -1 && GetBalance(root.Right) <= 0)
            return LeftRotate(root);

        if (balance < -1 && GetBalance(root.Right) > 0)
        {
            root.Right = RightRotate(root.Right);
            return LeftRotate(root);
        }

        return root;
    }

    private Node GetMinValueNode(Node node)
    {
        Node current = node;
        while (current.Left != null)
            current = current.Left;

        return current;
    }
    // Ağacın yazdırılması (in-order)
    public void PrintTree()
    {
        PrintTreeRec(root);
        Console.WriteLine();
    }

    private void PrintTreeRec(Node node)
    {
        if (node != null)
        {
            PrintTreeRec(node.Left);
            Console.Write(node.Value + " ");
            PrintTreeRec(node.Right);
        }
    }

    public void BalanceTree()
    {
        root = BalanceRec(root);
    }

    private Node BalanceRec(Node node)
    {
        if (node == null)
            return null;

        node.Left = BalanceRec(node.Left);
        node.Right = BalanceRec(node.Right);

        node.Height = Math.Max(GetHeight(node.Left), GetHeight(node.Right)) + 1;

        int balance = GetBalance(node);
    }

```

```

    if (balance > 1)
    {
        if (GetBalance(node.Left) < 0)
            node.Left = LeftRotate(node.Left);
        return RightRotate(node);
    }

    if (balance < -1)
    {
        if (GetBalance(node.Right) > 0)
            node.Right = RightRotate(node.Right);
        return LeftRotate(node);
    }

    return node;
}

static void Main(string[] args)
{
    AVLTree tree = new AVLTree();
    bool continueRunning = true;

    while (continueRunning)
    {
        Console.Clear();
        Console.WriteLine("1. Ekleme");
        Console.WriteLine("2. Silme");
        Console.WriteLine("3. Arama");
        Console.WriteLine("4. Listele");
        Console.WriteLine("5. Çıkış");
        Console.Write("Seçiminizi yapın: ");
        string choice = Console.ReadLine();

        switch (choice)
        {
            case "1":
                Console.Write("Eklenecek değeri girin: ");
                int insertValue = int.Parse(Console.ReadLine());
                tree.Insert(insertValue);
                Console.WriteLine($"Değer {insertValue} ağaçta eklendi.");
                tree.PrintTree();
                break;

            case "2":
                Console.Write("Silinecek değeri girin: ");
                int deleteValue = int.Parse(Console.ReadLine());
                tree.Delete(deleteValue);
                Console.WriteLine($"Değer {deleteValue} ağaçtan silindi.");
                tree.PrintTree();
                break;

            case "3":
                Console.Write("Aranacak değeri girin: ");
                int searchValue = int.Parse(Console.ReadLine());
                bool found = tree.Search(searchValue);
                if (found)
                    Console.WriteLine($"Değer {searchValue} ağaçta bulundu.");
                else
                    Console.WriteLine($"Değer {searchValue} ağaçta bulunamadı.");
        }
    }
}

```

```
        break;

    case "4":
        Console.WriteLine("Ağaç: ");
        tree.PrintTree();
        break;

    case "5":
        continueRunning = false;
        break;

    default:
        Console.WriteLine("Geçersiz seçim. Tekrar deneyin.");
        break;
}

Console.WriteLine("Devam etmek için bir tuşa basın...");
Console.ReadKey();
}
}
}
```